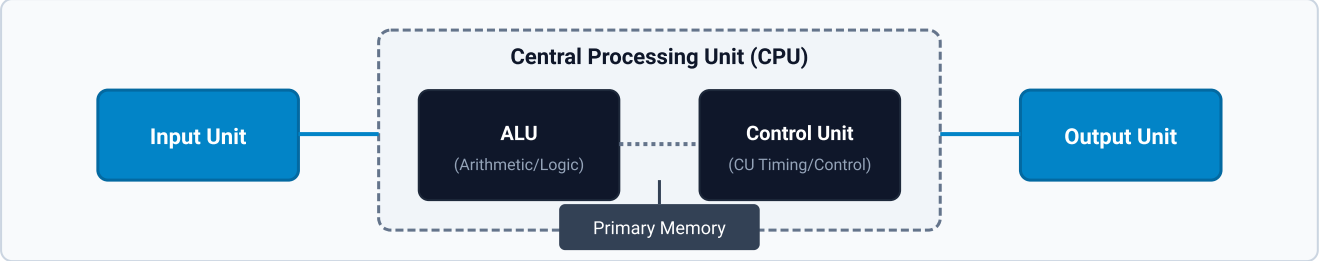
AKTU B.TECH SEM 3: COA

Unit 1: Computer Basics & Data Representation

KCS-302
Detailed Study Notes

1. FUNCTIONAL UNITS OF A DIGITAL COMPUTER

A computer system is an integrated set of functional units that processes binary data according to stored instructions. The hardware architecture consists of five primary functional units linked together by system buses.



2. VON NEUMANN VS. HARVARD ARCHITECTURE

The two foundational paradigms of computer organization dictate how memory and instruction execution are structured within the processor.

Architectural Feature	Von Neumann Architecture	Harvard Architecture
Memory Structure	Shared memory for data and instructions	Separate physical memories for data and instructions
Bus System	Single set of address/data buses	Separate data and instruction bus sets
Execution Speed	Slower (sequential fetch-execute bottleneck)	Faster (simultaneous instruction fetch & data access)
Hardware Complexity	Simple design, lower production cost	Complex bus routing and control mechanism
Bottleneck Problem	Suffers from *Von Neumann Bottleneck*	No bus conflict between fetch and data read/write
Application Area	General-purpose computers (PCs, Servers)	Microcontrollers, DSPs, Embedded Systems

The Von Neumann Bottleneck

Because data and instructions share the same physical bus in Von Neumann systems, the CPU cannot read an instruction and read/write data simultaneously. Throughput is strictly limited by bus transfer speeds compared to internal CPU processing rates.

3. COMPUTER BUS STRUCTURE & ARBITRATION

A bus is a collection of wires through which data, addresses, and control signals are transmitted between components. The system bus is classified into three functional lines:

- **Data Bus:** Bi-directional wires that transfer binary data between the CPU, memory, and I/O devices. Bus width (e.g., 32-bit, 64-bit) determines word size.
- **Address Bus:** Uni-directional lines carrying memory addresses from the CPU. Width determines maximum addressable memory (2^N bytes for N address lines).
- **Control Bus:** Signals managing read/write operations, interrupts, bus requests, and clock synchronization.

Bus Arbitration Mechanisms

Bus arbitration prevents conflicts when multiple bus masters (e.g., CPU, DMA controller) request control of the shared bus simultaneously:

- **Daisy Chaining (Centralized):** Serial connection. Requests pass through devices in priority order. Simple hardware, but vulnerable to single-point failure.
- **Centralized Parallel Arbitration:** A central bus controller receives individual request lines (**BR**) and issues grants (**BG**) based on priority algorithms.
- **Distributed Arbitration:** Devices independently determine priority using self-selection logic on shared arbitration lines without a central master.

4. FIXED-POINT DATA REPRESENTATION

Fixed-point representation assumes a fixed position for the binary point. Signed binary numbers are represented in three standard forms: Signed-Magnitude, 1's Complement, and 2's Complement.



N-1 +5 = 00000101 | -5 = 11111011

1's Complement Arithmetic

Invert all bits to form negatives. End-around carry is added back to the LSB upon arithmetic overflow during addition operations.

2's Complement Arithmetic

Formed by adding 1 to the 1's complement. Preferred in hardware because subtraction is simplified to simple addition (`A - B = A + (-B)`).

5. MULTIPLICATION ALGORITHM: BOOTH'S ALGORITHM

Booth's algorithm multiplies signed binary numbers in 2's complement notation. It reduces the number of additions required by scanning groups of consecutive 1s.

Booth's Multiplication Step-by-Step Procedure

1. Let Multiplicand = M, Multiplier = Q, Accumulator $A = 0$, and extra bit $Q_{-1} = 0$.
2. Set Sequence Counter $SC = N$ (where N is the bit-length of operands).
3. Inspect the least significant bit of multiplier Q_0 and Q_{-1}:
   - **00 or 11:** No operation. Perform Arithmetic Right Shift (ARS) on $[A, Q, Q_{-1}]$.
   - **10:** Subtract multiplicand from accumulator ($A = A - M$), then perform ARS on $[A, Q, Q_{-1}]$.
   - **01:** Add multiplicand to accumulator ($A = A + M$), then perform ARS on $[A, Q, Q_{-1}]$.
4. Decrement $SC = SC - 1$. Repeat step 3 until $SC = 0$.
5. Final combined register pair $[A, Q]$ contains the product result.

6. DIVISION ALGORITHMS: RESTORING & NON-RESTORING

Fixed-point division divides an $2N$-bit dividend Q by an N-bit divisor M to produce a quotient Q and remainder A.

Restoring Division Algorithm

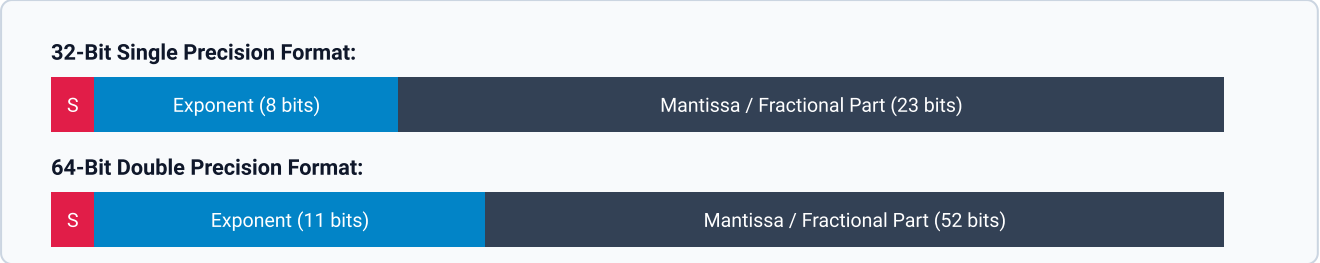
1. Shift $[A, Q]$ left 1 position.
2. Subtract divisor: $A = A - M$.
3. If sign bit of A is 1 (negative), set $Q_0 = 0$ and **restore** $A = A + M$.
4. If sign bit of A is 0 (positive), set $Q_0 = 1$.
5. Decrement SC. Repeat until $SC = 0$.

Non-Restoring Division Algorithm

Avoids redundant restore additions:
1. If $A \geq 0$, shift $[A, Q]$ left and $A = A - M$.
2. If $A < 0$, shift $[A, Q]$ left and $A = A + M$.
3. If $A \geq 0$, set $Q_0 = 1$; else set $Q_0 = 0$.
4. If final $A < 0$ at $SC=0$, restore $A = A + M$.

7. IEEE 754 FLOATING-POINT REPRESENTATION STANDARD

Real values are expressed in normalized floating-point notation: $V = (-1)^S \times (1 + Mantissa) \times 2^{(Exponent - Bias)}$.



Format Type	Total Bits	Sign Bit	Exponent Bits	Mantissa Bits	Bias Value
Single Precision	32 bits	1 bit [31]	8 bits [30-23]	23 bits [22-0]	127
Double Precision	64 bits	1 bit [63]	11 bits [62-52]	52 bits [51-0]	1023

8. FLOATING-POINT ARITHMETIC OPERATIONS

Floating-point addition and subtraction require alignment of binary points prior to performing significand computations:

1. **Check for Zeros:** If one operand is zero, return the other operand immediately.
2. **Align Exponents:** Compare exponents E_1 and E_2. Shift the mantissa of the smaller operand right by $|E_1 - E_2|$ bits until exponents match.
3. **Add/Subtract Mantissas:** Perform significand addition/subtraction based on sign bits.
4. **Normalize Result:** Shift result mantissa left/right to restore the implicit leading bit format $1.F$, adjusting exponent accordingly.
5. **Round & Overflow Check:** Apply IEEE rounding mode and test for exponent underflow/overflow condition.

9. AKTU SOLVED NUMERICAL EXAMPLES

Example 1: Multiply (+7) by (-3) using Booth's Algorithm (4-bit)

Operands: Multiplicand $\mathbf{M = +7 = 0111}$, $\mathbf{-M = 1001}$. Multiplier $\mathbf{Q = -3 = 1101}$, $\mathbf{Q_{-1} = 0}$, $\mathbf{A = 0000}$, $\mathbf{SC = 4}$.

Iteration	A	Q	Q_{-1}	Operation	SC
Initial	0000	1101	0	Initialize registers	4
Step 1	1001 1100	1101 1110	0 1	$\mathbf{Q_0Q_{-1} = 10 \rightarrow A = A - M}$ ARS [A, Q, Q_{-1}]	3
Step 2	0101 0010	1110 1111	1 0	$\mathbf{Q_0Q_{-1} = 01 \rightarrow A = A + M}$ ARS [A, Q, Q_{-1}]	2
Step 3	1011 1101	1111 1111	0 1	$\mathbf{Q_0Q_{-1} = 10 \rightarrow A = A - M}$ ARS [A, Q, Q_{-1}]	1
Step 4	1110	1111	1	$\mathbf{Q_0Q_{-1} = 11 \rightarrow No\ Op}$, ARS	0

Result: $\mathbf{[A, Q] = 1110\ 1111_2}$. In 2's complement = $\mathbf{-21_{10}}$. (Correct!)

Example 2: Convert $(-12.625)_{10}$ to IEEE 754 Single Precision Format

1. **Sign Bit:** Number is negative, so $\mathbf{S = 1}$.
2. **Binary Conversion:** $\mathbf{12_{10} = 1100_2}$ and $\mathbf{0.625_{10} = 0.101_2}$. Thus $\mathbf{12.625_{10} = 1100.101_2}$.
3. **Normalize:** Move decimal 3 positions left: $\mathbf{1.100101 \times 2^3}$.
   - Exponent $\mathbf{E = 3}$.
   - Mantissa fraction $\mathbf{F = 100101000...}$ (padded with zeros to 23 bits).
4. **Biased Exponent:** $\mathbf{E' = E + 127 = 3 + 127 = 130_{10} = 10000010_2}$.
5. **Final 32-Bit Representation:**
   $\mathbf{1\ |\ 10000010\ |\ 10010100000000000000000}$
   Hexadecimal equivalent: $\mathbf{C14A0000_{16}}$.